



Quantathon CR 2026 · Challenge 2

ESPAÑOL · (Challenge 2)

Challenge 2: Hacia el agua limpia para todos (predecir si una muestra de agua es potable)

Descripción (preview)

Este reto ataca uno de los problemas más urgentes del mundo: el acceso a agua potable. Vas a entrenar un clasificador que, a partir de mediciones fisicoquímicas del agua (pH, conductividad, turbidez, sólidos disueltos), prediga si una muestra es potable o no. Lo interesante es que compararás una Máquina de Vectores de Soporte (SVM) clásica contra una versión cuántica (QSVM) que usa kernels cuánticos, explorando si los mapas de características cuánticos capturan la estructura de los datos de forma distinta. Usarás el dataset Water Potability de Kaggle, construirás la matriz de kernel cuántico en el emulador H2 de Quantinuum y compararás ambos modelos en cinco métricas. Nivel accesible-intermedio: ideal si manejas fundamentos de machine learning clásico. La ventaja cuántica no es el objetivo; el objetivo es construir intuición real sobre kernels cuánticos. Conecta con el ODS 6.

ODS: ODS 6

Dificultad: Accesible-Intermedio

Algoritmos: QSVM (kernels cuánticos)

Formación recomendada: Conocimientos básicos de ML clásico

Contexto e impacto

El acceso al agua potable segura sigue siendo uno de los desafíos más urgentes que enfrenta la humanidad. El ODS 6 exige el acceso universal y equitativo al agua potable segura y asequible para 2030; sin embargo, miles de millones de personas todavía carecen de acceso confiable a agua que cumpla estándares básicos de calidad. El agua contaminada causa enfermedades prevenibles (cólera, tifoidea, disentería) que afectan de manera desproporcionada a comunidades de bajos ingresos y a niños menores de cinco años.

El aprendizaje automático ofrece una vía escalable para la detección automatizada de la calidad del agua. Mediciones fisicoquímicas como el pH, la conductividad, la turbidez y los sólidos disueltos pueden recolectarse económicamente en campo y alimentar un clasificador que señale las muestras no potables. En este desafío, los participantes comparan una SVM clásica frente a una QSVM que usa métodos de kernel cuántico, explorando si los mapas de características cuánticos pueden codificar los datos de forma que complemente o eventualmente supere a los

enfoques clásicos. La ventaja cuántica no es el objetivo: el ejercicio desarrolla intuición práctica sobre kernels cuánticos, una de las primitivas de ML cuántico a corto plazo con mayor probabilidad de uso real.

Requisitos previos y presupuesto de tiempo

- Conocimientos requeridos: Python, fundamentos de scikit-learn, álgebra lineal. Es útil (no obligatorio) familiaridad con la SVM clásica.
- Lectura previa sugerida (2-3 horas): tutorial de kernels cuánticos; Schuld & Killoran (2019).
- Día 1: preparación de datos + línea base de SVM clásica (5-6 h).
- Día 2: construcción del kernel cuántico + entrenamiento de QSVM (6-8 h).
- Día 3: comparación, extensiones opcionales, informe (5-6 h).

Fundamento científico

Una SVM clasifica datos encontrando el hiperplano de margen máximo en un espacio de características inducido por un kernel. Dada $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$, el problema de optimización es: minimizar $\frac{1}{2} \|w\|^2 + C \sum (x_i)$ sujeto a $y_i (w^T \phi(x_i) + b)$ mayor o igual a $1 - x_i$, con x_i mayor o igual a 0. Un kernel cuántico reemplaza el kernel clásico con el producto interno al cuadrado de dos estados cuánticos preparados por un circuito parametrizado: $K_Q(x_i, x_j) = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$. El mapa de características $\phi(x)$ codifica cada punto en un estado cuántico; la matriz K se pasa a un SVC clásico (scikit-learn, kernel='precomputed'). El circuito define la geometría del límite de decisión mientras la optimización sigue siendo clásica.

Motivación cuántica

Los mapas de características cuánticos pueden acceder a espacios de Hilbert exponencialmente grandes, representando correlaciones que los kernels clásicos solo aproximan de forma implícita. Para datos fisicoquímicos de alta dimensión, pueden capturar términos de interacción (por ejemplo pH x conductividad) de forma más natural que kernels polinomiales o RBF.

Resultado suficientemente bueno: QSVM alcanza F1 mayor o igual a 0.60 en el conjunto de prueba reservado, y la matriz de kernel muestra estructura no trivial (no casi uniforme).

Línea base clásica

Una SVM con kernel RBF, ajuste de hiperparámetros por validación cruzada de 5 particiones: C en $\{0.1, 1, 10\}$, γ en $\{\text{scale}, \text{auto}, 0.01\}$. Reporte exactitud, precisión, exhaustividad (recall), F1 y matriz de confusión. Referencia: Cortes & Vapnik (1995).

Planteamiento del problema

1. Preparar el dataset Water Potability (Kaggle) con imputación por mediana, estandarización y balanceo de clases.
2. Entrenar una línea base SVM-RBF clásica con ajuste de hiperparámetros validado de forma cruzada.
3. Construir un mapa de características cuántico y calcular la matriz de kernel K .
4. Entrenar una QSVM usando K y evaluar en el mismo conjunto de prueba reservado.
5. Comparar ambos clasificadores en las cinco métricas; visualizar y analizar la estructura del kernel.

6. (Opcional) Ejecutar en hardware cuántico real o aplicar mitigación de ruido y reportar el impacto.

Consejo: use un subconjunto de 16-64 muestras para el experimento cuántico (la matriz completa $N \times N$ crece como $O(N^2)$ ejecuciones de circuito). Documente la estrategia de selección y preserve el balance de clases.

Dataset y preparación (Parte 1)

Water Potability de Kaggle (kaggle.com/datasets/adityakadiwal/water-potability). Nueve características fisicoquímicas (pH, dureza, sólidos, cloraminas, sulfato, conductividad, carbono orgánico, trihalometanos, turbidez); objetivo binario (0 = no potable, 1 = potable).

- Imputar valores faltantes (pH: 491 NaN; Sulfato: 781 NaN; Trihalometanos: 162 NaN) por mediana por clase.
- Estandarizar todas las características a media cero y varianza unitaria.
- Balancear clases con submuestreo o SMOTE (división original: aprox. 60% no potable, 40% potable).
- División estratificada 80/20. Seleccionar subconjunto balanceado de 16-64 muestras para el experimento cuántico y documentar la estrategia.

Línea base clásica (Parte 2)

Entrenar una SVM clásica con kernel RBF, que sirve como modelo de referencia contra el cual se compara la QSVM. Realizar el ajuste de hiperparámetros por validación cruzada de 5 particiones: C en {0.1, 1, 10}, gamma en {scale, auto, 0.01}. Reportar todos los resultados en el conjunto de prueba reservado:

Métrica	Requerido
Exactitud	Sí
Precisión	Sí
Exhaustividad (recall)	Sí
Puntuación F1	Sí
Matriz de confusión	Sí

Kernel cuántico (Parte 3)

Cree un mapa de características $\phi(x)$. Calcule la matriz de kernel $K_{ij} = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$ y pásela al SVC de scikit-learn con kernel='precomputed'. Entregables: diagrama del circuito, mapa de calor de la matriz de kernel, explicación del cálculo y del número de qubits.

Estudio de mapas de características (Parte 4)

Compare múltiples mapas (ZZFeatureMap, PauliFeatureMap y uno personalizado) para estudiar cómo la incrustación afecta la geometría del kernel. Cada codificador induce una geometría distinta: dos observaciones muy similares bajo un mapa pueden volverse casi ortogonales bajo otro. Un buen mapa asigna observaciones de la misma clase a estados con solapamiento grande, y de clases distintas a estados con solapamiento menor, produciendo una matriz con estructura de bloques y alineación con las etiquetas. Mayor expresividad no garantiza mejor separación: un mapa demasiado expresivo puede producir un kernel cercano a la identidad o casi uniforme, con poca información útil.

Evalúe cada mapa con: exactitud, exactitud balanceada, F1 y validación cruzada; alineación kernel-objetivo; similitudes intra e interclase; distribución y mapa de calor del kernel; espectro de valores propios y rango efectivo; profundidad del circuito tras transpilación; número de compuertas de dos qubits; sensibilidad al muestreo finito, al ruido y a topologías de entrelazamiento; y costo computacional. Los experimentos de ablación varían repeticiones, entrelazamiento, términos de Pauli, escalado de características y capas de recarga de datos.

Extensiones opcionales

- A - Ejecución en hardware: ejecutar el cálculo del kernel en un backend real; comparar con el simulador y analizar el ruido.
- B - Mitigación de ruido: aplicar ZNE y mitigación de errores de lectura; reportar la mejora en la fidelidad del kernel.

Limitaciones honestas

- Conjunto de datos pequeño: el subconjunto (16-64 muestras) es demasiado chico para conclusiones estadísticas.
- Número limitado de qubits: restringe la expresividad de los mapas.
- Efectos del ruido: corrompe las entradas del kernel y degrada el clasificador en QPUs reales.
- Sin ventaja cuántica demostrada: las SVM-RBF superan a QSVM en todos los tamaños factibles hoy.
- Sobrecarga: leer la matriz completa requiere $O(N^2)$ ejecuciones.

Herramientas y recursos requeridos

- Pytket, Guppy.
- SVC de scikit-learn con kernel='precomputed'.
- Dataset: kaggle.com/datasets/adityakadiwal/water-potability.
- Referencias: Schuld & Killoran (2019); Havlicek et al. (2019), Nature; Cortes & Vapnik (1995).

Errores comunes

- Fuga de datos: el subconjunto cuántico se extrae solo del conjunto de entrenamiento; nunca incluya muestras de prueba.
- Características sin normalizar: los mapas cuánticos son sensibles a la escala; estandarice antes de codificar.
- Semidefinición positiva: un kernel ruidoso puede no ser PSD; regularice (suma epsilon I) si el SVC no converge.
- Comparación injusta: use las mismas divisiones y métricas para ambos clasificadores.
- Una sola ejecución: reporte media y desviación estándar de 3 o más ejecuciones.

Rúbrica de evaluación (Challenge 2)

Criterio	Excelente (4)	Bueno (3)	Necesita mejorar (1-2)
Línea base clásica	SVM-RBF con validación cruzada de 5 particiones sobre la cuadrícula completa; las cinco métricas; matriz de confusión incluida	SVM entrenada pero con ajuste limitado o métricas faltantes	Sin línea base clásica o implementación incorrecta

Construcción del kernel cuántico	Mapa bien fundamentado; matriz calculada y visualizada; diagrama de circuito; explicación de Kij con estudio de mapas y análisis cuantitativo	Kernel funcional; visualización o explicación limitada	Kernel incorrecto o circuito faltante
Clasificación QSVM	Comparación lado a lado de todas las métricas; análisis reflexivo; limitaciones reconocidas	Comparación presente pero con análisis superficial	Sin comparación o errores significativos
Extensiones opcionales	Ejecución en hardware O mitigación de ruido	Se intentó alguna extensión	Sin extensiones
Conexión con el ODS 6	Vínculo explícito con monitoreo real de calidad del agua con cadena causal; considera escala, costo e implementación	Mención general de la relevancia del agua limpia	Sin conexión con los ODS

Plataforma y stack (todos los challenges)

Para ejecutar los algoritmos cuánticos tendremos disponible el emulador H2 de Quantinuum, que permite el tratamiento exacto de hasta 26 qubits.

Stack de trabajo: los equipos usarán el software toolkit de Quantinuum: Pytket y/o Guppy.

QEC (eje transversal, opcional)

La QEC (corrección de errores cuánticos) es un eje transversal opcional. Para el Challenge 2 (QML), la codificación QEC combinada con circuitos ya profundos es poco viable a escala de hackathon. La mitigación de errores (ZNE, mitigación de lectura) es la opción más pragmática aquí.

Requisitos de entrega (todos los challenges)

Cada entrega debe incluir:

- Un repositorio público de GitHub con todo el código (en Pytket y/o Guppy), un requirements.txt, un único script o notebook de punto de entrada que reproduzca cada figura y cifra reportada, y un README.md.
- Un informe técnico escrito (PDF, máx. 8 páginas) con planteamiento del problema, línea base clásica, resumen de la implementación cuántica, resultados con barras de error y una sección honesta de limitaciones (obligatorio).
- Una presentación de 5 minutos con diapositivas.
- Una breve declaración (menor o igual a 200 palabras) sobre el lenguaje y SDK elegidos: qué funcionó, qué no y qué faltó.

El incumplimiento del requisito de reproducibilidad genera deducciones en todos los criterios de la rúbrica.

Criterios generales de evaluación

Todos los equipos son evaluados con la misma rúbrica, conforme a la evaluación técnica de la OQI. Los jueces premian el rigor y la honestidad por encima de la ambición: un resultado modesto, bien delimitado y totalmente reproducible puntúa más alto que una afirmación impresionante que no se pueda verificar.

Criterio	Descripción	Peso
Línea base clásica	Referencia de rendimiento válida de una fuente publicada, citada claramente. La comparación debe hacerse frente al método clásico más fuerte disponible.	15%
Implementación cuántica	Algoritmo correctamente implementado en el SDK elegido; resultados válidos; código limpio, documentado y reproducible. Las extensiones opcionales cuentan positivamente.	30% (Intento 10% / Buena ejecución 10% / Ejecución en hardware cuántico real 10%)
Comparación y escalado	Comparación directa cuántico vs. clásico en la misma instancia; escalado en 2 o más tamaños de problema; extrapolación honesta.	20%
Impacto en los ODS	Submeta específica del ODS identificada; cadena causal hacia un resultado real articulada; 2 o más ODS adicionales considerados.	5%
Reproducibilidad	El código corre desde un entorno limpio usando requirements.txt; un único punto de entrada reproduce todos los resultados.	10%
Explicación	Los participantes pueden dar una explicación técnica coherente de cómo funciona su código.	20%

Orientación para los jueces

- Sin sistema de niveles: los tres challenges compiten en un único grupo con la misma rúbrica. Un Challenge 1 impecable puede puntuar más alto que un Challenge 3 que exagera resultados.
- Ejecución sobre ambición: una ejecución perfecta del núcleo puede puntuar igual o más alto que extensiones incompletas.
- La honestidad se premia: limitaciones claras, barras de error y comparaciones clásicas honestas superan a afirmaciones cuánticas exageradas.
- Red flags: afirmaciones de 'ventaja cuántica' sin comparación de escalado; ausencia de la sección de limitaciones; cherry-picking de la mejor ejecución; resultados de hardware sin análisis de ruido; código que no corre limpio.

ENGLISH · (Challenge 2)

Challenge 2: Towards clean water for all (predicting whether a water sample is potable)

Description (preview)

This challenge tackles one of the most urgent problems in the world: access to safe drinking water. You will train a classifier that, from physicochemical water measurements (pH, conductivity, turbidity, dissolved solids), predicts whether a sample is potable or not. The interesting part is that you will compare a classical Support Vector Machine (SVM) against a quantum version (QSVM) that uses quantum kernels, exploring whether quantum feature maps capture the structure of the data differently. You will use the Water Potability dataset from Kaggle, build the quantum kernel matrix on Quantinuum's H2 emulator, and compare both models across five metrics. Accessible-intermediate level: ideal if you know classical machine-learning basics. Quantum advantage is not the goal; the goal is to build real intuition about quantum kernels. It connects to SDG 6.

SDG: SDG 6

Difficulty: Accessible-Intermediate

Algorithms: QSVM (quantum kernels)

Recommended background: Basic knowledge of classical ML

Context and impact

Access to safe drinking water remains one of the most urgent challenges facing humanity. SDG 6 calls for universal and equitable access to safe and affordable drinking water by 2030, yet billions of people still lack reliable access to water that meets basic quality standards. Contaminated water causes preventable diseases (cholera, typhoid, dysentery) that disproportionately affect low-income communities and children under five.

Machine learning offers a scalable path to automated water-quality screening. Physicochemical measurements such as pH, conductivity, turbidity, and dissolved solids can be collected cheaply in the field and fed into a classifier that flags non-potable samples. In this challenge, participants compare a classical SVM against a QSVM using quantum kernel methods, exploring whether quantum feature maps can encode the data in ways that complement or eventually surpass classical approaches. Quantum advantage is not the goal: the exercise builds practical intuition about quantum kernels, one of the near-term quantum ML primitives most likely to find real-world use.

Prerequisites and time budget

- Required knowledge: Python, scikit-learn basics, linear algebra. Familiarity with classical SVM is helpful but not required.
- Suggested pre-reading (2-3 hours): quantum kernel tutorial; Schuld & Killoran (2019).

- Day 1: data preparation + classical SVM baseline (5-6 h).
- Day 2: quantum kernel construction + QSVM training (6-8 h).
- Day 3: comparison, optional extensions, report (5-6 h).

Scientific background

An SVM classifies data by finding the maximum-margin hyperplane in a kernel-induced feature space. Given $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$, the optimization is: minimize $\frac{1}{2} \|w\|^2 + C \sum(x_i)$ subject to $y_i (w^T \phi(x_i) + b)$ greater than or equal to $1 - x_i$, with x_i greater than or equal to 0. A quantum kernel replaces the classical kernel with the squared inner product of two quantum states prepared by a parameterized circuit: $K_Q(x_i, x_j) = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$. The feature map $\phi(x)$ encodes each point into a quantum state; the kernel matrix K is passed to a classical SVC (scikit-learn, kernel='precomputed'). The circuit defines the geometry of the decision boundary while the optimization stays classical.

Quantum motivation

Quantum feature maps can access exponentially large Hilbert spaces, potentially representing correlations that classical kernels only approximate implicitly. For high-dimensional physicochemical data, they may capture interaction terms (e.g. pH x conductivity) more naturally than polynomial or RBF kernels.

Good-enough result: QSVM achieves F1 greater than or equal to 0.60 on the held-out test set, and the kernel matrix shows non-trivial structure (not nearly uniform).

Classical baseline

An RBF-kernel SVM, hyperparameter tuning via 5-fold cross-validation: C in $\{0.1, 1, 10\}$, γ in $\{\text{scale}, \text{auto}, 0.01\}$. Report accuracy, precision, recall, F1, and confusion matrix. Reference: Cortes & Vapnik (1995).

Problem statement

11. Prepare the Water Potability dataset (Kaggle) with median imputation, standardization, and class balancing.
12. Train a classical RBF-SVM baseline with cross-validated hyperparameter tuning.
13. Construct a quantum feature map and compute the quantum kernel matrix K .
14. Train a QSVM using K and evaluate on the same held-out test set.
15. Compare both classifiers across all five metrics; visualize and analyze kernel structure.
16. (Optional) Run on real quantum hardware or apply noise mitigation and report the impact.

Tip: use a subset of 16-64 samples for the quantum experiment (the full $N \times N$ matrix grows as $O(N^2)$ circuit executions). Document the subset selection strategy and preserve the class balance.

Dataset and preparation (Part 1)

Water Potability from Kaggle (kaggle.com/datasets/adityakadiwal/water-potability). Nine physicochemical features (pH, hardness, solids, chloramines, sulfate, conductivity, organic carbon, trihalomethanes, turbidity); binary target (0 = non-potable, 1 = potable).

- Impute missing values (pH: 491 NaN; Sulfate: 781 NaN; Trihalomethanes: 162 NaN) with per-class median.
- Standardize all features to zero mean and unit variance.
- Balance classes with undersampling or SMOTE (original split: about 60% non-potable, 40% potable).
- Stratified 80/20 split. Select a balanced subset of 16-64 samples for the quantum experiment and document the strategy.

Classical baseline (Part 2)

Train a classical SVM with an RBF kernel, which serves as the reference model against which the QSVM is compared. Perform hyperparameter tuning via 5-fold cross-validation: C in {0.1, 1, 10}, gamma in {scale, auto, 0.01}. Report all results on the held-out test set:

Metric	Required
Accuracy	Yes
Precision	Yes
Recall	Yes
F1-score	Yes
Confusion matrix	Yes

Quantum kernel (Part 3)

Create a feature map $\phi(x)$. Compute $K_{ij} = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$ and pass it to scikit-learn's SVC with kernel='precomputed'. Deliverables: circuit diagram, kernel matrix heatmap, explanation of the computation and qubit count.

Feature map study (Part 4)

Compare multiple feature maps (ZZFeatureMap, PauliFeatureMap, and a custom one) to study how the embedding affects kernel geometry. Each encoder induces a different geometry: two very similar observations under one map can become nearly orthogonal under another. A good map sends same-class observations to states with large overlap and different-class observations to states with smaller overlap, producing a block-structured matrix aligned with the labels. Greater expressivity does not guarantee better separation: an over-expressive map may produce a kernel close to the identity or nearly uniform, with little useful information.

Evaluate each map with: accuracy, balanced accuracy, F1, and cross-validation; kernel-target alignment; within-class and between-class similarities; kernel distribution and heatmap; eigenvalue spectrum and effective rank; circuit depth after transpilation; two-qubit gate count; sensitivity to finite sampling, noise, and entanglement topologies; and computational cost. Ablations vary repetitions, entanglement, Pauli terms, feature scaling, and data-reuploading layers.

Optional extensions

- A - Hardware execution: run the kernel computation on a real backend; compare with the simulator and analyze noise.
- B - Noise mitigation: apply ZNE and readout error mitigation; report the improvement in kernel fidelity.

Honest limitations

- Small dataset: the subset (16-64 samples) is too small for statistical conclusions.
- Limited qubit counts: restrict feature map expressivity.
- Noise effects: corrupt kernel entries and degrade the classifier on real QPUs.
- No demonstrated quantum advantage: RBF-SVMs outperform QSVM at all currently feasible sizes.
- Kernel overhead: reading the full matrix requires $O(N^2)$ circuit executions.

Required tools and resources

- Guppy, Pytket.
- scikit-learn SVC with kernel='precomputed'.
- Dataset: kaggle.com/datasets/adityakadiwal/water-potability.
- References: Schuld & Killoran (2019); Havlicek et al. (2019), Nature; Cortes & Vapnik (1995).

Common pitfalls

- Subset leakage: draw the quantum subset from the training set only; never include test samples.
- Unnormalized features: quantum feature maps are scale-sensitive; standardize before encoding.
- Positive semi-definiteness: a noisy kernel may not be PSD; regularize (add epsilon I) if the SVC fails to converge.
- Unfair comparison: use identical splits and metric definitions for both classifiers.
- Single run: report mean and standard deviation across 3 or more runs.

Judging rubric (Challenge 2)

Criterion	Excellent (4)	Good (3)	Needs Work (1-2)
Classical baseline	RBF-SVM with 5-fold CV across the full hyperparameter grid; all five metrics; confusion matrix included	SVM trained but limited tuning or missing metrics	No classical baseline or incorrect implementation
Quantum kernel construction	Well-motivated map; kernel computed and visualized; circuit diagram; explanation of Kij with feature-map study and quantitative analysis	Working kernel; limited visualization or explanation	Incorrect kernel or missing circuit
QSVM classification	Side-by-side comparison of all metrics; thoughtful analysis; limitations acknowledged	Comparison present but shallow analysis	No comparison or significant errors
Optional extensions	Hardware execution OR noise mitigation	Some extension attempted	No extensions
SDG 6 connection	Explicit link to real-world water-quality monitoring with causal chain; considers scale, cost, deployment	General mention of clean-water relevance	No SDG connection

Platform and stack (all challenges)

To run the quantum algorithms we will have available Quantinuum's H2 emulator, which allows the exact treatment of up to 26 qubits.

Working stack: teams will use Quantinuum's software toolkit: Pytket and/or Guppy.

QEC (transversal axis, optional)

QEC (quantum error correction) is an optional transversal axis. For Challenge 2 (QML), QEC encoding combined with already-deep circuits is unlikely to be feasible at hackathon scale. Error mitigation (ZNE, readout mitigation) is the more pragmatic choice here.

Submission requirements (all challenges)

Every submission must include:

17. A public GitHub repository containing all code (in Pytket and/or Guppy), a requirements.txt, a single entry-point script or notebook that reproduces every figure and number reported, and a README.md.
18. A written technical report (PDF, max 8 pages) with problem framing, classical baseline, quantum implementation summary, results with error bars, and an honest limitations section (required).
19. A 5-minute presentation with slides.
20. A short statement (200 words or less) on the software toolkit: what worked, what did not, and what was missing.

Failure to meet the reproducibility requirement results in deductions across all rubric criteria.

General judging criteria

All teams are evaluated on the same rubric, following the OQI technical evaluation. Judges reward rigour and honesty over ambition: a modest, well-scoped, fully reproducible result will score higher than an impressive-sounding claim that cannot be verified.

Criterion	Description	Weight
Classical baseline	Valid performance reference from a published source, cited clearly. Benchmarking must be against the strongest available classical method.	15%
Quantum implementation	Algorithm correctly implemented in Pytket and/or Guppy; valid results; clean, documented, reproducible code. Optional extensions count positively.	30% (Attempt 10% / Good execution 10% / Run on real quantum hardware 10%)
Benchmarking and scaling	Direct comparison of quantum vs. classical on the same instance; scaling across 2 or more problem sizes; honest extrapolation.	20%
SDG impact	Specific SDG sub-target identified; causal chain to a real-world outcome articulated; 2 or more other SDGs considered.	5%
Reproducibility	Code runs from a clean	10%

	environment using requirements.txt; a single entry point reproduces all results.	
Explanation	Participants can give a coherent technical explanation of how their code works.	20%

Guidance for judges

- No tier system: all three challenges compete in a single pool on the same rubric. An impeccable Challenge 1 can score higher than a Challenge 3 that overstates results.
- Execution over ambition: perfect core execution can score equally or higher than incomplete extensions.
- Honesty is rewarded: clear limitations, error bars, and honest classical comparisons beat overstated quantum claims.
- Red flags: 'quantum advantage' claims without a scaling comparison; missing honest-limitations section; cherry-picked best run; hardware results without noise analysis; code that does not run clean.